

1. ИНСТРУМЕНТ СБОРКИ MAVEN

Apache Maven — это инструмент автоматизации сборки проектов на основе концепции проекта (Project Object Model, POM), который помогает разработчикам управлять зависимостями, конфигурацией сборки и жизненным циклом проекта. Maven упрощает управление проектами, обеспечивая стандартизацию процесса сборки и управления зависимостями. Использование Maven делает проекты более переносимыми и легко поддерживаемыми.

В удобной среде разработки выбираем «Создать новый проект», в настройках нового проекта стоит выбрать Build System «Maven» (рисунок 9.1), по желанию поменять GroupId и ArtefactId.

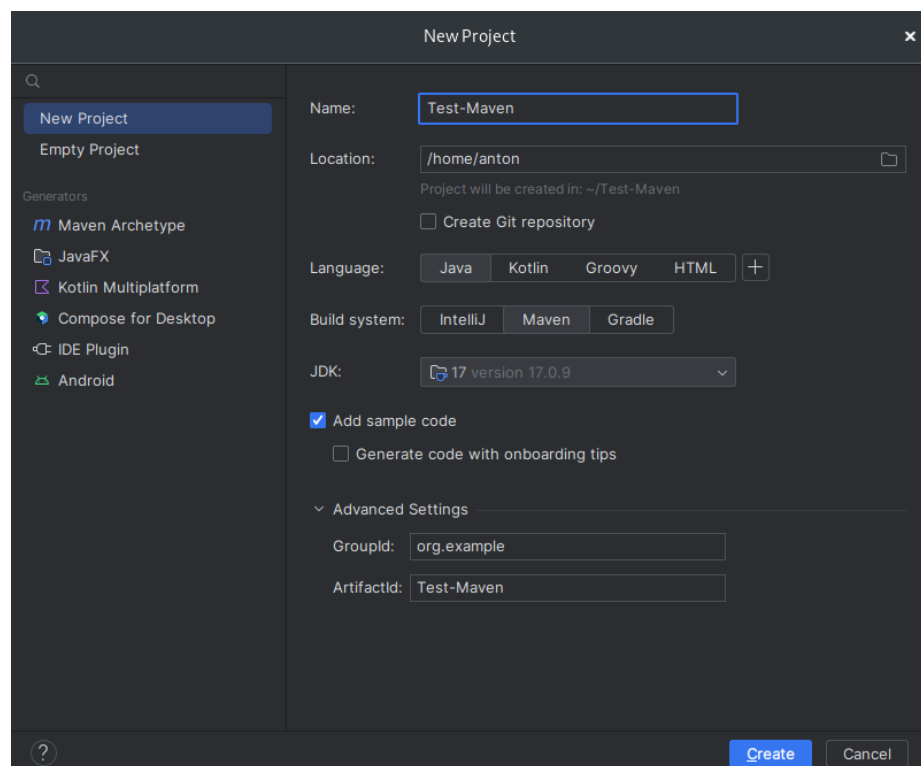


Рисунок 9.1. Создание Maven проекта

Далее нажимаем «Create». После этого видим базовую структуру проекта, подобную изображенной на рисунке 9.2.

За сборку проекта при использовании Maven отвечает файл `pom.xml`, который обычно располагается в корне проекта. Что из себя представляет данный файл? Рассмотрим пример такого файла.

Информация для программного проекта, поддерживаемого Maven, содержится в XML-файле с именем **pom.xml**. При исполнении Maven проверяет прежде всего, содержит ли этот файл все необходимые данные и все ли данные синтаксически правильно записаны.

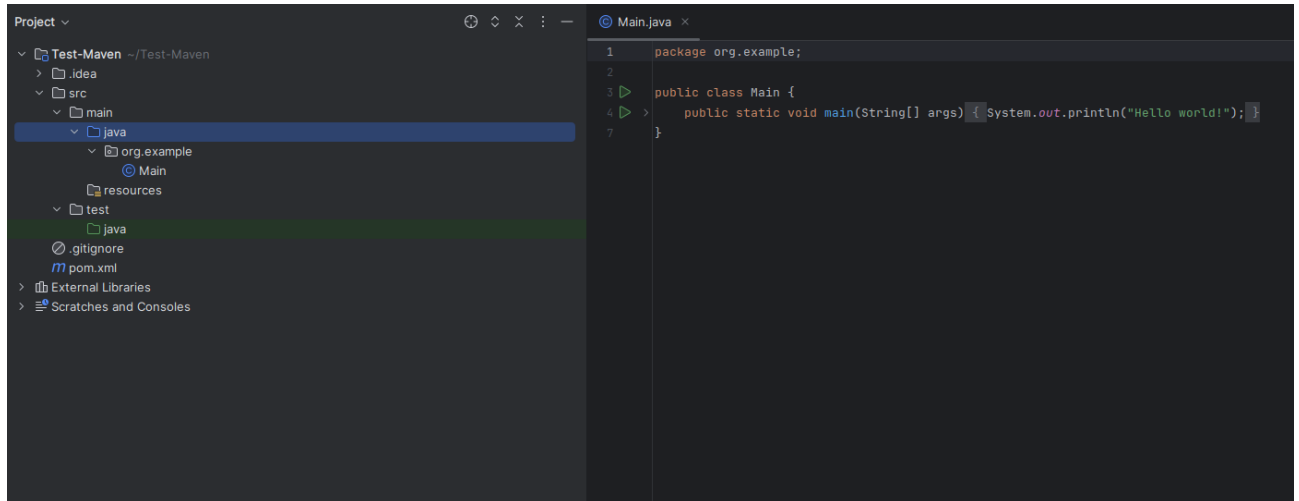


Рисунок 9.2. Созданный Maven проект и его структура

Пример файла **pom.xml**:

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <project xmlns="http://maven.apache.org/POM/4.0.0"
3.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5.     <modelVersion>4.0.0</modelVersion>
6.     <properties>
7.         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
8.     </properties>
9.
10.    <groupId>com.examclouds</groupId>
11.    <artifactId>courses</artifactId>
12.    <version>1.0-SNAPSHOT</version>
13.
14.    <dependencies>
15.        <dependency>
16.            <groupId>mysql</groupId>
17.            <artifactId>mysql-connector-java</artifactId>
18.            <version>5.1.39</version>
19.        </dependency>
20.
21.        <dependency>
22.            <groupId>com.h2database</groupId>
23.            <artifactId>h2</artifactId>
24.            <version>1.4.196</version>
25.        </dependency>
26.    </dependencies>
27.
28.    <build>
29.        <sourceDirectory>src</sourceDirectory>
30.        <resources>
```

```

31.         <resource>
32.             <directory>resources</directory>
33.         </resource>
34.     </resources>
35.     <plugins>
36.         <plugin>
37.             <groupId>org.apache.maven.plugins</groupId>
38.             <artifactId>maven-compiler-plugin</artifactId>
39.             <version>2.0.2</version>
40.             <configuration>
41.                 <source>1.8</source>
42.                 <target>1.8</target>
43.                 <encoding>UTF-8</encoding>
44.             </configuration>
45.         </plugin>
46.         <plugin>
47.             <groupId>org.apache.maven.plugins</groupId>
48.             <artifactId>maven-checkstyle-plugin</artifactId>
49.             <version>2.17</version>
50.             <configuration>
51. <suppressionsLocation>suppressions.xml</suppressionsLocation>
52.             </configuration>
53.         </plugin>
54.     </plugins>
55. </build>
56. <reporting>
57.     <plugins>
58.         <plugin>
59.             <groupId>org.apache.maven.plugins</groupId>
60.             <artifactId>maven-compiler-plugin</artifactId>
61.             <version>2.0.2</version>
62.             <configuration>
63.                 <source>1.8</source>
64.                 <target>1.8</target>
65.                 <encoding>UTF-8</encoding>
66.             </configuration>
67.         </plugin>
68.         <plugin>
69.             <groupId>org.apache.maven.plugins</groupId>
70.             <artifactId>maven-checkstyle-plugin</artifactId>
71.             <version>2.17</version>
72.             <configuration>
73. <suppressionsLocation>suppressions.xml</suppressionsLocation>
74.             </configuration>
75.         <reportSets>
76.             <reportSet>
77.                 <reports>
78.                     <report>checkstyle</report>
79.                 </reports>
80.             </reportSet>
81.         </reportSets>
82.     </plugin>
83.     <plugin>
84.         <groupId>org.apache.maven.plugins</groupId>
85.         <artifactId>maven-pmd-plugin</artifactId>
86.         <version>3.8</version>
87.     </plugin>

```

```

88.         <plugin>
89.             <groupId>org.codehaus.mojo</groupId>
90.             <artifactId>findbugs-maven-plugin</artifactId>
91.             <version>3.0.4</version>
92.             <configuration>
93.                 <xmlOutput>true</xmlOutput>
94.             </configuration>
95.         </plugin>
96.     </plugins>
97. </reporting>
98. </project>

```

1. Корневой элемент

Корневой элемент `<project>`, в котором прописана схема, облегчающая редактирование и проверку, и версия POM.

Пример 2. Корневой элемент

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <project xmlns="http://maven.apache.org/POM/4.0.0"
3.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5.     ...
6. </project>
7.

```

2. Заголовок

Внутри тега *project* содержится основная и обязательная информация о проекте:

Пример 3. Заголовок

```

1. <groupId>com.examclouds</groupId>
2. <artifactId>courses</artifactId>
3. <version>1.0-SNAPSHOT</version>

```

В *Maven* каждый проект идентифицируется парой *groupId*, *artifactId*.

Во избежание конфликта имён, **groupId** - наименование организации или подразделения и обычно действуют такие же правила, как и при именовании пакетов в Java - записывают доменное имя организации или сайта проекта.

artifactId - название проекта.

Внутри тега **version** хранится версия проекта.

Тройкой *groupId*, *artifactId*, *version* (далее - **GAV**) можно однозначно идентифицировать *jar* файл приложения или библиотеки. Если состояние кода для проекта не зафиксировано, то в конце к имени версии добавляется *"-SNAPSHOT"* что обозначает, что версия в разработке и результирующий *jar* файл может меняться.

3. Тег *packaging*

Тег *<packaging>* определяет какого типа файл будет создаваться как результат сборки. Возможные варианты *pom*, *jar*, *war*, *ear*.

Тег является необязательным. Если его нет, используется значение по умолчанию - *jar*.

4. Описание проекта

Также добавляется информация, которая не используется самим *Maven*, но нужна для программиста, чтобы понять, о чём этот проект:

Пример 4. Описание проекта

<name>powermock-core</name> название проекта для человека

<description>PowerMock core functionality.</description>

Описание проекта

<url>http://www.powermock.org</url> сайт проекта

5. Зависимости

Зависимости - следующая очень важная часть *pom.xml* - тут хранится список всех библиотек (зависимостей), которые используются в проекте. Каждая библиотека идентифицируется так же, как и сам проект — тройкой *groupId*, *artifactId*, *version* (GAV). Объявление зависимостей заключено в теге *<dependencies>...</dependencies>*.

Кроме GAV при описании зависимости может присутствовать тег *<scope>*. Он задаёт, для чего библиотека используется. В данном примере говорится, что библиотека с GAV *junit:junit:4.4* нужна только для выполнения тестов.

Пример 4. Зависимости

```
1. <dependencies>
2.   <dependency>
3.     <groupId>mysql</groupId>
4.     <artifactId>mysql-connector-java</artifactId>
5.     <version>5.1.39</version>
6.   </dependency>
7.
8.   <dependency>
9.     <groupId>com.h2database</groupId>
10.    <artifactId>h2</artifactId>
11.    <version>1.4.196</version>
12.  </dependency>
13. </dependencies>
```

6. Тег <build>

Тег *<build>* не обязательный, так как существуют значения по умолчанию. Этот раздел содержит информацию по самой сборке:

- где находятся исходные файлы,
- где находятся ресурсы,
- какие плагины используются.

Пример 5. Тег <build>

<sourceDirectory> - определяет, откуда *Maven* будет брать файлы исходного кода. По умолчанию это *src/main/java*, но вы можете определить, где это вам удобно. Директория может быть только одна (без использования специальных плагинов).

<resources> и вложенные в неё теги *<resource>* определяют, одну или несколько директорий, где хранятся файлы ресурсов. Ресурсы в отличие от файлов исходного кода при сборке просто копируются. Директория по умолчанию *src/main/resources*.

<outputDirectory> - определяет, в какую директорию компилятор будет сохранять результаты компиляции - **.class* файлы. Значение по умолчанию - *target/classes*.

<finalName> - имя результирующего *jar* (*war*, *ear* ...) файла с соответствующим типом расширением, который создаётся на фазе *package*. Значение по умолчанию — *artifactId-version*.

Maven плагины позволяют задать дополнительные действия, которые будут выполняться при сборке. Например, в приведённом примере добавлен плагин, который автоматически делает проверку кода на наличие "плохого" кода и потенциальных ошибок.

Это лишь пример некоторого готового файла `pom.xml`. Вернемся к нашему проекту. И добавим в него плагин для сборки, который поможет нам указать `Main Class` сразу из кода, чтоб не прописывать это при запуске. Итоговый файл `pom.xml` представлен на листинге 9.1.

Листинг 9.1. Файл `pom.xml`

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <project xmlns="http://maven.apache.org/POM/4.0.0"
3.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5.     <modelVersion>4.0.0</modelVersion>
6.
7.     <groupId>org.example</groupId>
8.     <artifactId>Test-Maven</artifactId>
9.     <version>1.0-SNAPSHOT</version>
10.
11.     <properties>
12.         <maven.compiler.source>17</maven.compiler.source>
13.         <maven.compiler.target>17</maven.compiler.target>
14.         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15.     </properties>
16.
17.     <build>
18.         <plugins>
19.             <plugin>
20.                 <groupId>org.codehaus.mojo</groupId>
21.                 <artifactId>exec-maven-plugin</artifactId>
22.                 <version>3.1.0</version>
23.                 <configuration>
24.                     <mainClass>org.example.Main</mainClass>
25.                     <skip>>false</skip>
26.                 </configuration>
27.             </plugin>
28.         </plugins>
29.     </build>
30. </project>
31.
```

Далее нам нужно проделать запуск проекта. Это можно сделать как из среды разработки, так и из системной консоли. Для запуска сборки проекта на базе *Maven* используется команда `mvn`. В системах на базе *Linux* `mvn` можно

установить, как `sudo apt install maven`, где `apt` — это ваш пакетный менеджер в Linux. Затем нам нужно из консоли переместиться в папку проекта (рисунок 9.3).

Чтоб создать команду запуска нам нужно определить фазы для запуска. Первой фазой будет `clean`, чтоб очистить директорию сборки, если проект уже собирался. Обычно `clean` используется для сборки «начистую». Вторая фаза будет `install`, в этой фазе компилируется исходный код, выполняются тесты и упаковываются результаты в `jar` или `war` файл, после чего упакованный файл копируется в локальный репозиторий `Maven`, что позволяет использовать его как зависимость в других проектах на этой же машине. Третья фаза `exec:java`, которая требуется для запуска приложения. Итоговая команда: `mvn clean install exec:java`. Запуск представлен на рисунке 9.4.

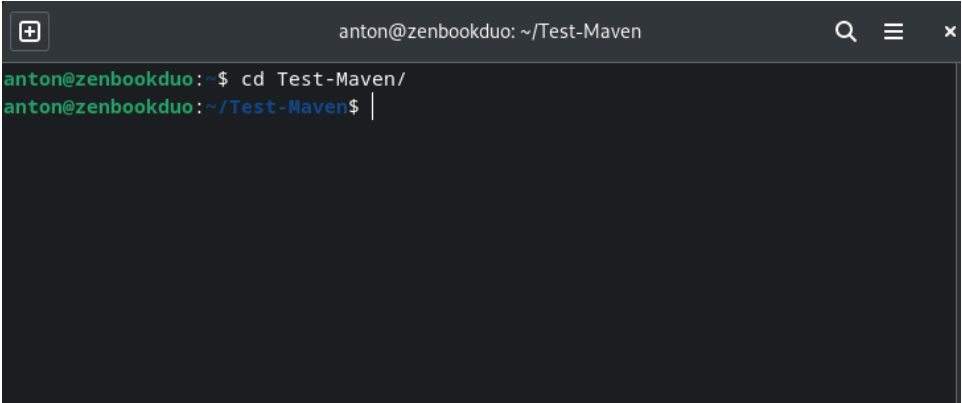
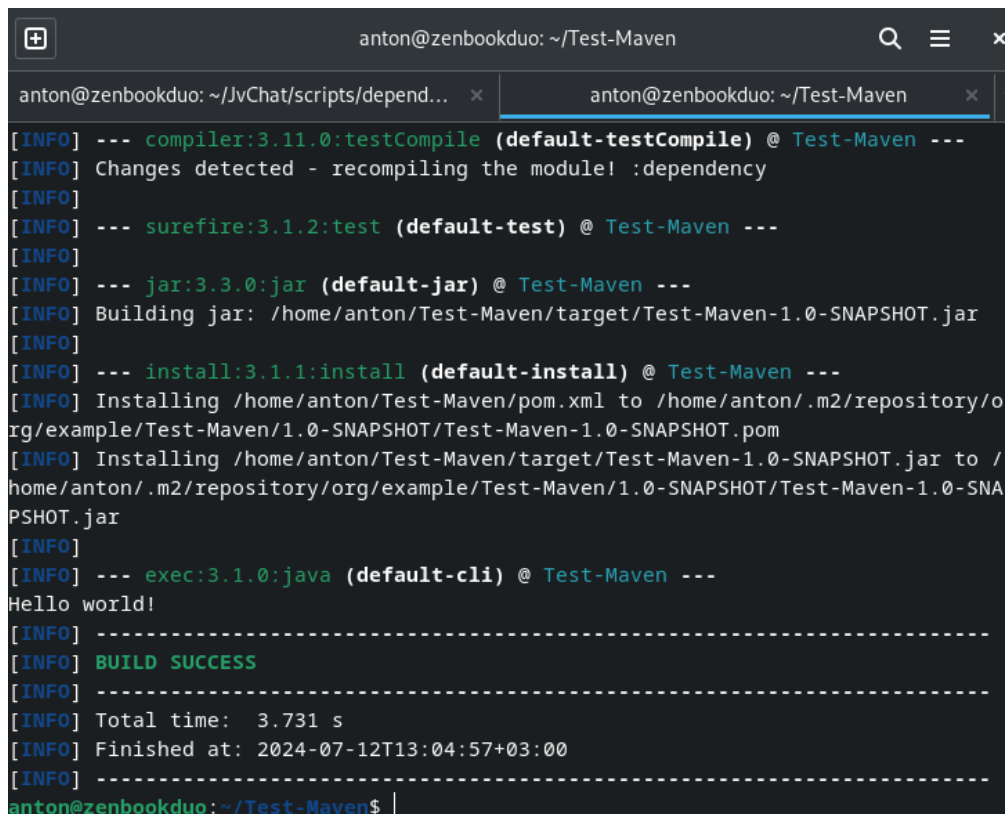
A screenshot of a terminal window with a dark background. The title bar at the top shows 'anton@zenbookduo: ~/Test-Maven' on the left and search, menu, and close icons on the right. The terminal content shows two lines: the first line is 'anton@zenbookduo:~\$ cd Test-Maven/' with the prompt in green and the command in white; the second line is 'anton@zenbookduo:~/Test-Maven\$' with the prompt in green and a white cursor at the end.

Рисунок 9.3. Перемещение в папку проекта

A terminal window titled 'anton@zenbookduo: ~/Test-Maven' showing the output of a Maven build. The output includes several [INFO] messages: '--- compiler:3.11.0:testCompile (default-testCompile) @ Test-Maven ---', 'Changes detected - recompiling the module! :dependency', '--- surefire:3.1.2:test (default-test) @ Test-Maven ---', '--- jar:3.3.0:jar (default-jar) @ Test-Maven ---', 'Building jar: /home/anton/Test-Maven/target/Test-Maven-1.0-SNAPSHOT.jar', '--- install:3.1.1:install (default-install) @ Test-Maven ---', 'Installing /home/anton/Test-Maven/pom.xml to /home/anton/.m2/repository/org/example/Test-Maven/1.0-SNAPSHOT/Test-Maven-1.0-SNAPSHOT.pom', 'Installing /home/anton/Test-Maven/target/Test-Maven-1.0-SNAPSHOT.jar to /home/anton/.m2/repository/org/example/Test-Maven/1.0-SNAPSHOT/Test-Maven-1.0-SNAPSHOT.jar', '--- exec:3.1.0:java (default-cli) @ Test-Maven ---', 'Hello world!', 'BUILD SUCCESS', 'Total time: 3.731 s', 'Finished at: 2024-07-12T13:04:57+03:00'. The prompt 'anton@zenbookduo: ~/Test-Maven\$' is visible at the bottom.

```
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ Test-Maven ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] --- surefire:3.1.2:test (default-test) @ Test-Maven ---
[INFO] --- jar:3.3.0:jar (default-jar) @ Test-Maven ---
[INFO] Building jar: /home/anton/Test-Maven/target/Test-Maven-1.0-SNAPSHOT.jar
[INFO] --- install:3.1.1:install (default-install) @ Test-Maven ---
[INFO] Installing /home/anton/Test-Maven/pom.xml to /home/anton/.m2/repository/org/example/Test-Maven/1.0-SNAPSHOT/Test-Maven-1.0-SNAPSHOT.pom
[INFO] Installing /home/anton/Test-Maven/target/Test-Maven-1.0-SNAPSHOT.jar to /home/anton/.m2/repository/org/example/Test-Maven/1.0-SNAPSHOT/Test-Maven-1.0-SNAPSHOT.jar
[INFO] --- exec:3.1.0:java (default-cli) @ Test-Maven ---
Hello world!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.731 s
[INFO] Finished at: 2024-07-12T13:04:57+03:00
[INFO] -----
anton@zenbookduo: ~/Test-Maven$
```

Рисунок 9.4. Консольный запуск сборки

Теперь рассмотрим второй способ сборки. В среде разработки выбираем «Редактировать конфигурации сборки», в разных средах эта кнопка может отличаться. Затем нажимаем «Добавить новую конфигурацию сборки», «Maven». После чего видим окошко похожее на изображенное на рисунке 9.5.

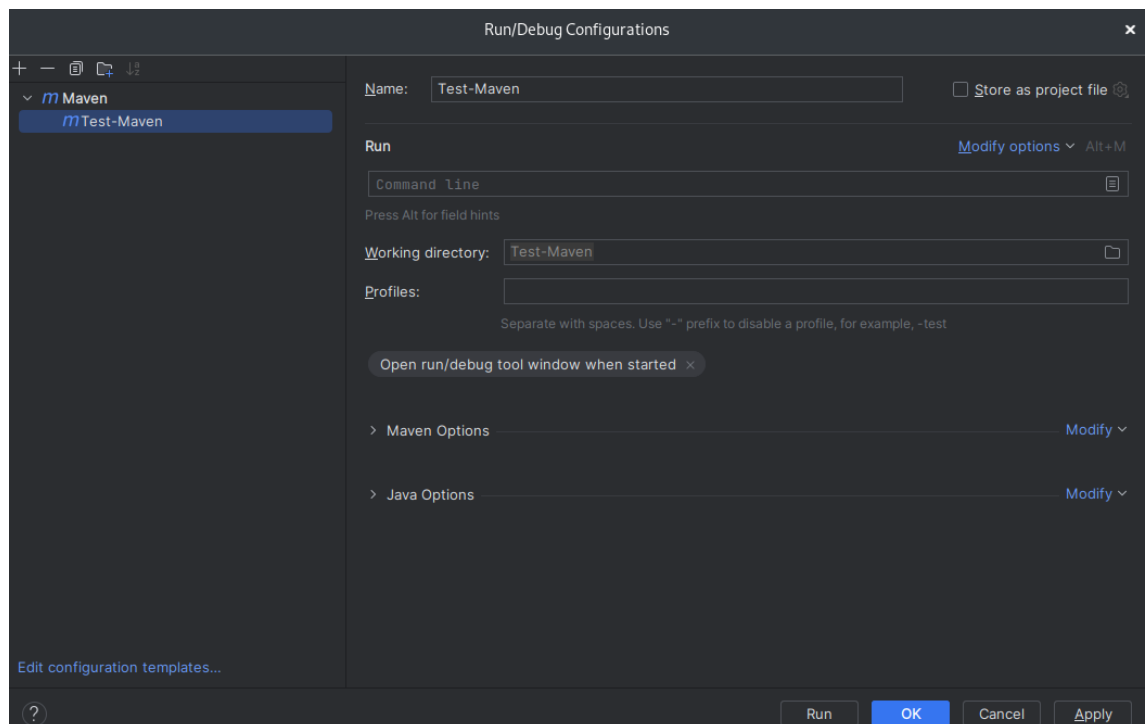


Рисунок 9.5. Окно создания новой конфигурации сборки

В этом окне надо в Command line занести наши фазы, а именно, `clean install exec:java` (рисунок 9.6). После чего нажимаем «Применить» и «Ок».

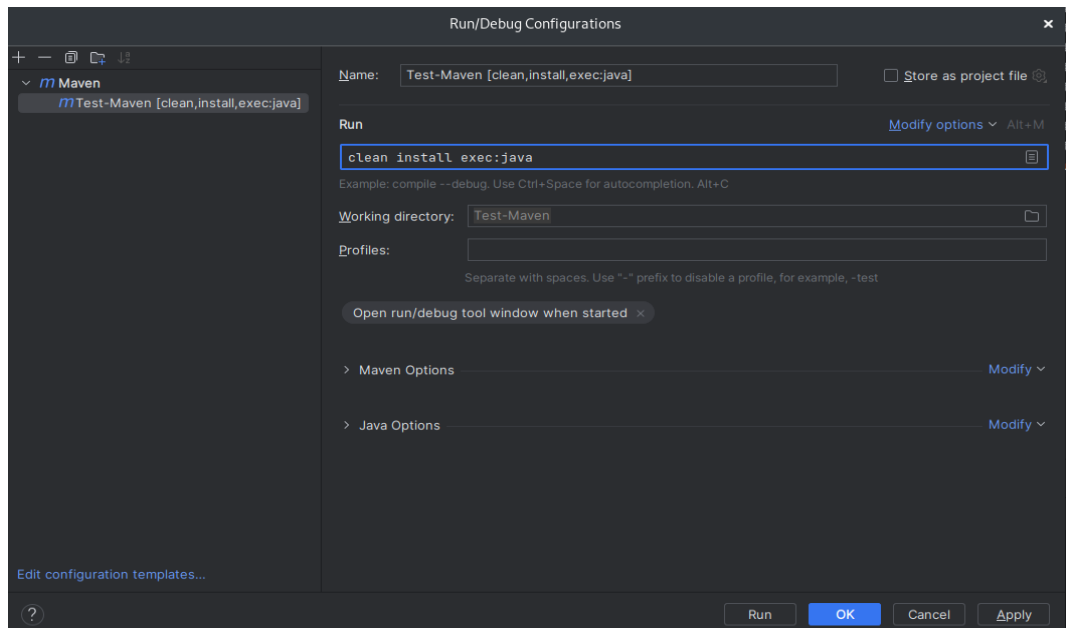


Рисунок 9.6. Готовая конфигурация Maven

После этого нажимаем на кнопку запуска конфигурации сборки в среде разработки. Результат запуска приведен на рисунке 9.7.

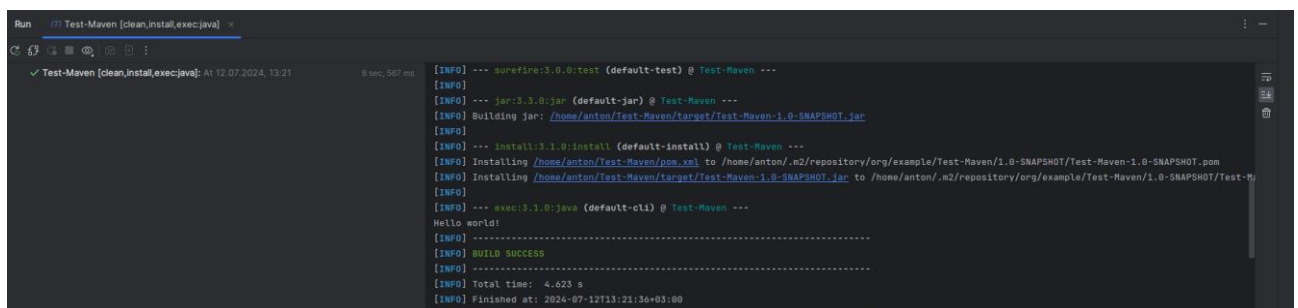


Рисунок 9.7 — Запуск конфигурации сборки

Задание для выполнения лабораторной работы

1. Создайте базовый проект и настройте `exec-maven-plugin` (код можно взять из любой лабораторной прошлого семестра или написать что-то новое).
2. Добавьте в проект зависимость для логирования (например, например, `Log4j` или `SLF4J`)

3. Модифицируйте класс Main для использования логирования вместо System.out
4. Добавьте зависимость для работы с JSON (например, Jackson)
5. Создайте класс для сериализации/десериализации объектов в JSON. Посмотрите какие транзитивные зависимости появились у проекта
6. Добавьте в секцию build плагин SpotBugs. Выполните команду: `mvn spotbugs:check`. Проанализируйте найденные проблемы и исправьте их.

Дополнительное задание: настройте работу плагина и сборки таким образом, чтобы если SpotBugs находит ошибки выше установленного порога (High/ Medium), то сборка завершается с ошибкой.

Контрольные вопросы

1. Что такое Apache Maven и для чего он используется?
2. Как установить Maven на различные операционные системы?
3. Какова структура проекта Maven?
4. Что такое POM файл и какова его роль в проекте Maven?
5. Какова структура файла POM?
6. Что такое зависимости (dependencies) в Maven и как они определяются?
7. Что такое репозиторий Maven и какие виды репозиториев существуют?
8. Как добавить зависимость в проект Maven?
9. Что такое плагины в Maven и как они используются?
10. Как создать новый проект Maven с помощью команды Maven?
11. Что такое цели (goals) и фазы (phases) в Maven и в чем их отличие?
12. Как выполнить команду сборки проекта в Maven?
13. Что такое жизненный цикл сборки (build lifecycle) в Maven?

14. Как настроить профили (profiles) в Maven для разных сред (например, разработка и продакшн)?
15. Как управлять версиями зависимостей в Maven?
16. Что такое "SNAPSHOT" версии в Maven и как они используются?
17. Как использовать Maven для создания отчета о качестве кода?
18. Какие команды Maven используются для очистки проекта, сборки, тестирования и установки?
19. Как интегрировать Maven с системой контроля версий, такой как Git?
20. Как добавить и настроить сторонний репозиторий в проекте Maven?
21. Какие скоупы зависимостей существуют в Maven и для чего они используются?
22. Чем отличается плагин от зависимости в Maven?
23. Как работает транзитивность зависимостей?
24. Для чего нужен плагин surefire?
25. Как исключить транзитивную зависимость?

